ORM 프레임워크 이해와 활용

01 ORM 개념 및 기본 원리

02 MyBatis 프레임워크 활용

03 ORM 프레임워크 사용 시 보안 이슈

📌 학습 목표

- ✅ ORM(Object-Relational Mapping) 프레임워크의 개념과 동작 방식을 이해한다.
- ✅ ORM 사용 시 발생할 수 있는 보안 취약점을 학습하고 보안 위협을 인식한다.
- ✅ ORM 인젝션을 방어하기 위한 보안 대책을 이해하고 적용할 수 있다.

📍 학습 내용

1. ORM 프레임워크의 개념 및 기본 원리
2. ORM 프레임워크의 장단점
3. MyBatis 프레임워크의 장단점
4. ORM 프레임워크에서 취약점이 발생하는 원인과 대책

ORM 프레임워크 이해와 활용

01 ORM 개념 및 기본 원리

02 MyBatis 프레임워크 활용

03 ORM 프레임워크 사용 시 보안 이슈

ORM(Object-Relational Mapping) 의 정의와 필요성

ORM의 정의

객체 지향 언어의 객체와 관계형 데이터베이스의 테이블 간 데이터를 자동으로 매핑하는 기술

ORM의 필요성

- 불일치 문제 해결: 객체 모델과 스키마 간 불일치 해소
- 개발 생산성 향상: SQL 직접 작성 부담 감소, 코드 간결화
- 유지보수성 및 보안 강화: 데이터베이스 접근 추상화로 보안 취약점(예: SQL Injection) 최소화

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name")
    private String name;

    @Column(name = "email")
    private String email;
}
```



id	name	email
guest	손님	NULL
admin	관리자	admin@corp.com
operator	운영자	op@corp.com

Table: users

전통적인 SQL 쿼리 vs ORM 방식 비교 – 정의와 특징

	전통적인 SQL 쿼리	ORM (Object-Relational Mapping)
정의	· 개발자가 SQL 문을 작성하여 데이터베이스와 상호작용 · 쿼리의 구조, 최적화, DBMS 고유 기능을 직접 활용 가능	· 객체지향 언어의 객체와 관계형 데이터베이스의 테이블을 매핑하는 기술 · 메서드 호출, 객체 조작을 통해 데이터베이스 작업 수행
특징	· 명령형(Imperative) 접근 방식 · 세밀한 제어와 복잡한 쿼리 작성에 유리	· 추상화 계층을 제공하여 코드 가독성과 유지보수성 향상 · DBMS 독립적 구현 가능





전통적인 SQL 쿼리 vs ORM 방식 비교 – 장점과 단점

	전통적인 SQL 쿼리	ORM (Object-Relational Mapping)
장점	· 복잡한 쿼리 작성 및 DBMS 최적화 기능 활용 가능 · 쿼리 튜닝 및 인덱스 활용에 직접 접근 가능 · 다양한 SQL 문법과 DB 기능에 바로 접근	· 객체 중심의 코드 작성으로 개발 속도 향상, 유지보수 용이 · DBMS 변경 및 플랫폼 독립적인 개발 지원 · 자동 파라미터 바인딩 등으로 SQL Injection 위험 감소
단점	· SQL 코드가 분산되어 있으면 관리 및 변경이 복잡 · 특정 DBMS에 특화된 문법 사용 시 이식성 저하 · 파라미터 처리 미흡 시 SQL Injection 위험	· 복잡한 쿼리나 대량 데이터 처리 시 최적화 한계 존재 · 복잡한 조인이나 세부 최적화가 어려워 추가 Raw SQL 필요 · ORM의 내부 동작과 설정을 이해해야 함

전통적인 SQL 쿼리 vs ORM 방식 비교

전통적 SQL 쿼리 방식이 적합한 경우

- 성능 튜닝이 중요한 경우
- 복잡한 다중 조인 쿼리 작성 시
- 특정 DBMS 고유 기능(예: 저장 프로시저, 윈도우 함수)을 적극 활용할 때



ORM 사용이 적합한 경우

- CRUD 중심의 애플리케이션, 빠른 프로토타이핑 및 유지보수가 중요한 경우
- 데이터베이스 독립성이 필요하거나 객체 지향 패러다임을 선호할 때



ORM이 제공하는 주요 기능

매핑 기능

- 객체-테이블 매핑
  클래스와 테이블, 필드와 컬럼을 자동으로 매핑하여 SQL 작성 없이 데이터 조작
- 관계 매핑
  1:1, 1:N, N:M 관계를 어노테이션 또는 XML 설정을 통해 정의 및 관리
- 자동 스키마 생성
  도메인 모델 변경에 따라 DB 스키마를 자동 생성하거나 업데이트 가능

CRUD 작업 지원

- Create, Read, Update, Delete 기능을 객체 조작만으로 처리
- 데이터베이스 직접 접근 없이, 객체 상태 변경에 따른 자동 동기화(DIRTY CHECKING)

ORM이 제공하는 주요 기능

고급 쿼리 기능

- 객체 지향 쿼리 언어 (HQL, JPQL 등)
  SQL보다 추상화된 방식으로 데이터 조회 및 조작 가능
- Criteria API
  프로그래밍 방식으로 동적 쿼리 작성 지원
- 네이티브 쿼리 지원
  필요시 직접 SQL을 실행할 수 있는 기능 제공

Lazy Loading & 캐싱

- 지연 로딩(Lazy Loading)
  실제 사용 시점에 연관 데이터를 로드하여 초기 로딩 비용 절감
- 1차/2차 캐시
  동일 세션 내 또는 애플리케이션 전역에서 캐시를 활용해 성능 최적화

ORM이 제공하는 주요 기능

트랜잭션 관리

- 자동 트랜잭션 처리
  명시적 트랜잭션 없이, ORM 프레임워크가 트랜잭션 경계를 관리
- 동시성 제어 및 낙관적/비관적 Lock 지원
  데이터 무결성을 유지하며 여러 사용자의 동시 접근 처리

기타 부가 기능

- 연관 관계 및 Fetch 전략
  연관 객체 로딩 전략 설정으로 성능 및 메모리 최적화
- 유연한 설정 및 확장성
  XML, 어노테이션, 플러그인 등을 통한 다양한 설정 옵션과 확장성 제공

ORM 프레임워크 이해와 활용

01 ORM 개념 및 기본 원리

02 MyBatis 프레임워크 활용

03 ORM 프레임워크 사용 시 보안 이슈

MyBatis 프레임워크 - 개요

MyBatis 프레임워크의 정의

MyBatis는 Java Persistence 프레임워크로, SQL 쿼리와 자바 객체 간의 매핑을 지원

개발자가 직접 SQL을 작성한 후, XML이나 어노테이션을 통해 매핑을 설정하여 DB와 상호작용

특징

· 복잡한 쿼리와 최적화에 유리

· ORM의 추상화 없이 직접 SQL 작성으로 세밀한 제어 가능

· XML, 어노테이션 등 다양한 방식으로 매핑 설정 가능



MyBatis 프레임워크 – 주요 기능

SQL 매핑 및 실행

· Mapper 파일(XML) 또는 어노테이션 기반으로 SQL 문을 정의

· SQL 문과 자바 객체 간의 자동 매핑 제공

동적 SQL 지원

조건문(if, choose) 및 반복문(foreach)을 사용하여 유연한 쿼리 작성 가능

타입 핸들러 (Type Handler)

자바 객체와 DB 데이터 간 변환을 위한 사용자 정의 타입 핸들러 지원

플러그인 기능

인터셉터를 통해 쿼리 실행 전/후 로직을 확장하거나 커스터마이징 가능

MyBatis 프레임워크 – 설정 및 구성

설정 파일 (mybatis-config.xml)

환경(데이터소스, 트랜잭션 매니저 등) 설정 및 전역 옵션 구성

Mapper 파일 구성

- SQL 문과 매핑 정보를 담은 XML 파일 또는 어노테이션 사용
- 네임스페이스를 통해 Mapper를 구분하고 재사용성 향상

데이터 소스 및 세션 관리

- DB 커넥션 풀과 **`SqlSessionFactory`**를 이용해 세션 생성 및 관리
- 스프링과 연동 시, 스프링 부트 자동 설정 및 통합 활용

MyBatis 프레임워크 사용 예시 (1/3)

설정 파일 (XML 형식)

- 데이터베이스 연결 정보
- 트랜잭션 관리
- Mapper 등록

```
<configuration>
    <environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/mydb"/>
                <property name="username" value="root"/>
                <property name="password" value="password"/>
            </dataSource>
        </environment>
    </environments>
    <mappers>
        <mapper resource="mapper/UserMapper.xml"/>
    </mappers>
</configuration>
```

MyBatis 프레임워크 사용 예시 (2/3)

XML 기반 매핑

```xml
<mapper namespace="com.example.mapper.UserMapper">
    <select id="getUserById" resultType="com.example.model.User">
        SELECT * FROM users WHERE id = #{id}
    </select>

    <insert id="insertUser">
        INSERT INTO users (name, email) VALUES (#{name}, #{email})
    </insert>

    <update id="updateUser">
        UPDATE users SET name = #{name}, email = #{email} WHERE id = #{id}
    </update>

    <delete id="deleteUser">
        DELETE FROM users WHERE id = #{id}
    </delete>
</mapper>
```

Java 어노테이션 기반

```java
public interface UserMapper {
    @Select("SELECT * FROM users WHERE id = #{id}")
    User getUserById(@Param("id") int id);

    @Insert("INSERT INTO users(name, email) VALUES(#{name}, #{email})")
    void insertUser(User user);

    @Update("UPDATE users SET name = #{name}, email = #{email} WHERE id = #{id}")
    void updateUser(User user);

    @Delete("DELETE FROM users WHERE id = #{id}")
    void deleteUser(@Param("id") int id);
}
```

MyBatis 프레임워크 사용 예시 (3/3)

동적 SQL 지원 (XML 형식)

- `<if>`
- `<where>`
- `<foreach>`

```
<select id="searchUsers" resultType="com.example.model.User">
    SELECT * FROM users
    <where>
        <if test="name != null">
            name = #{name}
        </if>
        <if test="email != null">
            AND email = #{email}
        </if>
    </where>
</select>
```

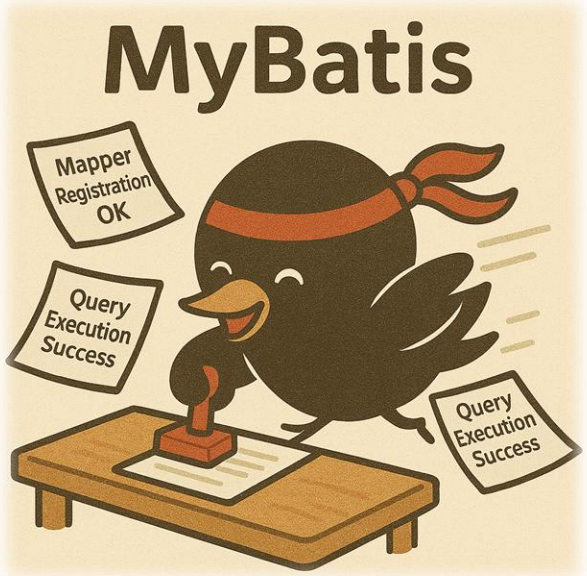
MyBatis 프레임워크 – 장단점 및 사례

장점

- SQL을 직접 작성하므로, 복잡한 쿼리 및 성능 튜닝에 유리
- ORM보다 간단한 설정으로 데이터 매핑을 관리
- DB 작업의 전 과정을 개발자가 직접 관리 가능

단점

- 직접 SQL 작성 시, 비슷한 쿼리의 중복 관리 필요
- ORM처럼 완전한 객체 관계 자동 매핑은 제공하지 않음
- XML/어노테이션 매핑 파일 관리 및 동적 SQL 작성에 대한 학습 필요



사용 사례

- 대규모 데이터 처리, 복잡한 쿼리 최적화가 요구되는 애플리케이션
- 기존 JDBC의 반복 작업을 줄이면서도 세밀한 SQL 제어가 필요한 프로젝트

ORM 프레임워크 이해와 활용

01 ORM 개념 및 기본 원리

02 MyBatis 프레임워크 활용

03 ORM 프레임워크 사용 시 보안 이슈

ORM 프레임워크와 보안 개요

ORM 프레임워크의 역할

- 객체와 관계형 데이터베이스 간 매핑을 자동화하여 개발 생산성을 높임
- SQL을 직접 작성하지 않고도 CRUD 및 복잡한 쿼리 수행 가능

보안 이슈의 중요성

- ORM 자체가 SQL Injection 등을 기본적으로 방어하지만, 잘못된 사용 시 취약점이 발생할 수 있음
- 매핑, 캐싱, 동적 쿼리 등 여러 요소가 보안에 영향을 미칠 수 있음

SQL 인젝션 및 동적 쿼리 취약점

SQL 인젝션 주의

기본적인 ORM은 파라미터 바인딩으로 보호되지만, 동적 쿼리를 문자열 결합 방식으로 작성하면 SQL 인젝션 발생할 수 있음

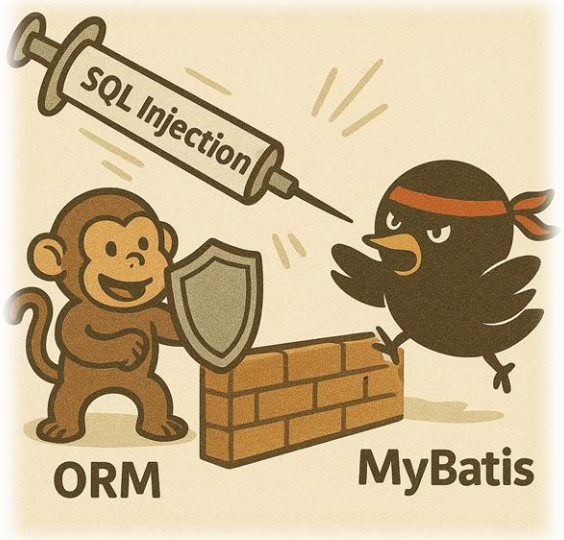
→ Native Query 사용 시 주의 필요

대응 방안

· 항상 안전한 파라미터 바인딩 방식을 사용해야 함

· 동적 쿼리 작성 시, ORM이 제공하는 동적 SQL 태그(예: MyBatis의 if, choose 등) 사용

· Native Query 사용 시 별도 검증 및 파라미터 바인딩 철저



캐싱 및 지연 로딩 관련 보안 이슈

캐싱 취약점

- 2차 캐시나 공유 캐시 설정 시 민감 데이터가 의도치 않게 노출될 가능성
- 캐시 무결성 관리 미흡 시, 캐시 변조 공격(Cache Poisoning) 위험

지연 로딩 및 데이터 노출

- 필요 이상으로 연관된 데이터가 지연 로딩되어, 예상치 못한 데이터 노출 발생 가능
- ORM 매핑 시 민감 정보가 포함된 엔티티를 불필요하게 로드하는 문제

대응 방안

- 캐시 설정 시 민감 데이터 제외 및 접근 제어 강화
- Lazy Loading 설정과 연관 데이터 조회 시 필요한 데이터만 선택적으로 로드하도록 설계

매핑, 설정 및 권한 관리 취약점

잘못된 매핑 설정

- 엔티티 매핑 오류로 인한 데이터 누락 또는 과도한 데이터 노출
- 양방향 매핑 및 연관 관계 설정 미흡 시, 무한 루프 또는 불필요한 데이터 로드 발생

Mass Assignment (대량 할당) 취약점

클라이언트 입력 값을 그대로 엔티티에 바인딩할 경우, 원치 않는 필드 변경 가능

대응 방안

- 엔티티 매핑 시, 민감 필드는 명시적으로 제외하거나 읽기 전용으로 설정
- 입력값 검증과 DTO(Data Transfer Object)를 활용하여, 대량 할당 공격 방지



요약 정리

- ORM은 객체와 관계형 데이터베이스 간의 매핑을 자동화하여 개발 생산성을 높임.
- SQL을 직접 작성하지 않아도 CRUD 및 복잡한 쿼리 수행 가능.
- SQL Injection 방어 기능이 있지만, 잘못된 설정 시 보안 취약점 발생 가능.
- MyBatis는 SQL을 직접 작성하면서도 매핑을 자동화하여, 복잡한 쿼리 최적화에 유리.
- ORM & MyBatis를 활용할 때, 보안 및 성능 최적화를 고려한 설계가 필수.

참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

감사합니다.



퀴즈

1. 다음 중 ORM(Object-Relational Mapping)의 주요 장점이 아닌 것은?

1) 객체와 RDB간의 매핑을 자동화하여 개발 생산성을 높일 수 있음
2) SQL을 직접 만들지 않아도 CRUD 및 복잡한 쿼리 수행이 가능
3) 모든 SQL 최적화를 자동으로 수행하여 성능 문제 없음
4) 데이터베이스 종속성을 줄이고 유지보수를 용이하게 함

2. MyBatis 프레임워크의 특징으로 적절한 것은?

1) SQL 작성하지 않고 객체 조작만으로 DB 작업을 수행
2) SQL을 직접 작성하면서도 매핑 작업을 줄여 생산성을 높임
3) 완전한 객체-관계 매핑으로 DB 구조를 자동으로 변경 가능
4) 기본적으로 동적 SQL을 지원하지 않음

3. ORM을 잘못 사용할 경우 발생할 수 있는 대표적인 보안 취약점 중 하나로, SQL 쿼리를 문자열 결합 방식으로 생성할 때 발생할 수 있는 공격 기법은 무엇인가?

( )



퀴즈 정답

1. **다음 중 ORM(Object-Relational Mapping)의 주요 장점이 아닌 것은?**

   3) 모든 SQL 최적화를 자동으로 수행하여 성능 문제 없음

   ORM은 자동 최적화 기능이 없으며, 복잡한 쿼리의 경우 성능 튜닝이 필요합니다.

2. **MyBatis 프레임워크의 특징으로 적절한 것은?**

   2) SQL을 직접 작성하면서도 매핑 작업을 줄여 생산성을 높임

   MyBatis는 ORM과 달리 SQL을 직접 작성해야 하지만, 매핑을 도와주는 기능을 제공하여 반복 작업을 줄이고 생산성을 높입니다.

3. **ORM을 잘못 사용할 경우 발생할 수 있는 대표적인 보안 취약점 중 하나로, SQL 쿼리를 문자열 결합 방식으로 생성할 때 발생할 수 있는 공격 기법은 무엇인가?**

   (SQL Injection 또는 SQL 인젝션)